

Fiche méthode : régression linéaire avec scikit-learn

Du jeu de données à l'importance des variables (exemple : prix de l'immobilier en Californie)

Étape	Fonctions / code	Pourquoi
1. Imports	<pre>import numpy as np, pandas as pd import plotly.express as px import plotly.graph_objects as go from sklearn.datasets import fetch_california_housing from sklearn.model_selection import train_test_split from sklearn.impute import SimpleImputer from sklearn.preprocessing import (StandardScaler, OneHotEncoder) from sklearn.compose import ColumnTransformer from sklearn.pipeline import Pipeline from sklearn.linear_model import LinearRegression from sklearn.metrics import r2_score</pre>	Tout charger une fois en haut du notebook. Pipeline enchaîne des étapes, ColumnTransformer applique un traitement différent selon le type de colonne.
2. Charger et mettre en DataFrame	<pre>data = fetch_california_housing() df = pd.DataFrame(data.data, columns=data.feature_names) df['Price'] = data.target df.head(); df.info(); df.describe() df.isna().sum()</pre>	Un DataFrame est plus simple à explorer et garde les noms de colonnes. info / describe / isna : types, échelles, valeurs manquantes (elles décident du preprocessing).
3. EDA (explorer)	<pre>px.scatter(df, x='MedInc', y='Price') px.scatter_matrix(df) corr = df.corr() px.imshow(corr, text_auto=True) # outliers : filtrer avec .quantile() q = df['AveRooms'].quantile(0.99) filtered = df[df['AveRooms'] < q]</pre>	Repérer quelles variables semblent liées à la cible, et les outliers qui déforment les graphiques et le modèle. Une corrélation mesure le lien avec UNE variable à la fois.
4. Séparer X et Y	<pre>X = df.drop('Price', axis=1) # features Y = df['Price'] # cible # baseline: X = df[['MedInc']] (2 crochets)</pre>	X = entrées du modèle, Y = ce qu'on prédit. Double crochets pour garder un DataFrame (2D), sinon scikit-learn plante.
5. Train / test split	<pre>X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)</pre>	Garder 20 % de données que le modèle ne voit jamais pour mesurer sa vraie performance. random_state rend le découpage reproductible. À faire AVANT tout preprocessing.
6. Preprocessing	<pre>num_cols = X_train.select_dtypes('number').columns num_pipe = Pipeline([('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())]) preprocessor = ColumnTransformer([('num', num_pipe, num_cols)]) X_train_p = preprocessor.fit_transform(X_train) X_test_p = preprocessor.transform(X_test)</pre>	Imputer : remplace les NaN. StandardScaler : moyenne 0, écart-type 1, ce qui rend les coefficients comparables. fit_transform sur train, transform seul sur test : on apprend moyenne/écart-type sur le train uniquement (sinon fuite de données). Variables catégorielles : ajouter ('cat', cat_pipe, cat_cols) avec OneHotEncoder.

Étape	Fonctions / code	Pourquoi
7. Entraîner	<pre>regressor = LinearRegression() regressor.fit(X_train_p, Y_train)</pre>	Le modèle apprend un coefficient par variable plus une constante (intercept). Toujours entraîner sur les données transformées du train.
8. Prédire et évaluer	<pre>Y_train_pred = regressor.predict(X_train_p) Y_test_pred = regressor.predict(X_test_p) print(r2_score(Y_train, Y_train_pred)) print(r2_score(Y_test, Y_test_pred))</pre>	R2 proche de 1 = bon, proche de 0 = mauvais. Comparer train et test : score train bien meilleur que test = surapprentissage ; les deux faibles = modèle trop simple.
9. Visualiser	<pre># 1 seule feature : nuage + droite fig = px.scatter(x=X_train['MedInc'], y=Y_train) fig.add_trace(go.Scatter(x=..., y=Y_train_pred)) # plusieurs features : réel vs prédit px.scatter(x=Y_test, y=Y_test_pred)</pre>	Avec plusieurs variables on ne peut plus tracer une droite ; on compare donc valeurs réelles et prédites (points proches de la diagonale = bon modèle).
10. Importance des variables	<pre>column_names = X_train.columns.tolist() coefs = pd.DataFrame(index=column_names, data=regressor.coef_.transpose(), columns=['coefficients']) imp = abs(coefs).sort_values('coefficients') fig = px.bar(imp, orientation='h') fig.update_layout(showlegend=False, margin=dict(l=150)) fig.show()</pre>	Les features étant standardisées, coefficient grand = variable influente. Le signe donne le sens (+ fait monter le prix, - le fait baisser). Les noms doivent être dans le même ordre que les colonnes passées au modèle.

Erreurs fréquentes et réflexes

- **AttributeError: ... has no attribute 'transformers_'** : cet attribut n'existe que sur un **ColumnTransformer** (après fit), pas sur un Pipeline ni sur un DataFrame/tableau. Pour boucler sur les colonnes num/cat, utiliser un ColumnTransformer ; sinon, `X_train.columns`.
- **fit_transform() renvoie un tableau numpy** : les noms de colonnes sont perdus. Stocker le résultat dans une **nouvelle variable** (`X_train_p`) et garder `X_train` en DataFrame.
- **ValueError: Shape of passed values is (8,1), indices imply (9,1)** : pas autant de noms que de coefficients (colonne Price restée dans X, liste gardée d'une ancienne exécution). Vérifier `len(column_names) == len(regressor.coef_)`.
- **Ne jamais fit sur le test** (scaler, imputer, encoder) : seulement transform.
- **Notebook qui se comporte bizarrement** : l'état des variables dépend de l'ordre d'exécution. Faire *Restart and Run All*.
- **Coefficients comparables seulement si les features sont standardisées**. Un coefficient mesure l'effet d'une variable toutes les autres étant fixées ; il peut donc différer de la corrélation simple avec la cible.

Squelette minimal à copier

```
X = df.drop('Price', axis=1); Y = df['Price']
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)
num_cols = X_train.select_dtypes('number').columns
preprocessor = ColumnTransformer([('num', Pipeline([('imputer', SimpleImputer(strategy='median')),
                                                    ('scaler', StandardScaler())]), num_cols)])

X_train_p = preprocessor.fit_transform(X_train)
X_test_p = preprocessor.transform(X_test)
regressor = LinearRegression().fit(X_train_p, Y_train)
print(r2_score(Y_train, regressor.predict(X_train_p)), r2_score(Y_test, regressor.predict(X_test_p)))
coefs = pd.DataFrame(regressor.coef_, index=X_train.columns, columns=['coefficients'])
px.bar(abs(coefs).sort_values('coefficients'), orientation='h').update_layout(showlegend=False)
```